

Exercise 5 Critical Sections, Race Conditions

Learning Goals:

- Recognize critical sections
- Recognize the consequences of a race condition
- Name countermeasures to prevent a race condition

Legend:

 Reading that must be completed before the exercise

 Questions/tasks to be completed before the exercise

 Tasks to be completed during the exercise

5.1 Preparations for the Exercise

The preparations and questions should be completed before the exercise session. The exception is this first exercise, for which you have not yet had an opportunity to read. They are also intended as aids to help you with the tasks to be completed during the exercise session.

 As preparation, study the following materials:

- Chap. 9.1 and 9.2 in Christian Baun, „Operating Systems (Bilingual Edition)”, Springer Vieweg

 **Introductory Questions:**

a) What is a critical section? Describe it briefly.

b) What is a race condition? Please describe with your own words.

- c) What are the advantages of signaling and waiting compared to polling?

- d) What is the difference between signaling and blocking?

- e) Go through the C program in Listing 1 given in Task 5.2 and describe the program behavior in a few sentences. You have already learned or will learn more about shared memory in detail. In the chapters 9.3.1 and 9.3.2 of Christian Baun, „Operating Systems“, Springer Vieweg you can find respective information.

- f) How do you interpret the following output for the given C program on race conditions? What is the cause of the calculation difference?

```
13 th child increased value by 53122044!  
15 th child increased value by 53222837!  
5 th child increased value by 52136941!  
3 th child increased value by 51710497!  
35 th child increased value by 52821782!  
22 th child increased value by 45804940!  
29 th child increased value by 53063602!  
0 th child increased value by 47694908!  
1 th child increased value by 53322091!  
12 th child increased value by 52272667!  
32 th child increased value by 52412523!  
6 th child increased value by 47881518!  
36 th child increased value by 48435708!  
23 th child increased value by 45743854!  
2 th child increased value by 46014072!  
4 th child increased value by 52252716!  
9 th child increased value by 46416827!  
38 th child increased value by 48156723!  
8 th child increased value by 45789772!  
17 th child increased value by 53287546!  
30 th child increased value by 53167480!  
33 th child increased value by 52548349!  
28 th child increased value by 53040796!  
11 th child increased value by 45741368!  
21 th child increased value by 48073369!  
20 th child increased value by 47778332!  
25 th child increased value by 46029036!  
31 th child increased value by 52345704!  
18 th child increased value by 52108931!  
14 th child increased value by 53526505!  
37 th child increased value by 52795836!  
16 th child increased value by 53295613!  
24 th child increased value by 46113229!  
27 th child increased value by 53424345!  
34 th child increased value by 48588345!  
7 th child increased value by 45802535!  
26 th child increased value by 45931101!  
10 th child increased value by 46083250!  
19 th child increased value by 48453551!  
39 th child increased value by 48449552!  
Shared M. = 100000001 - MAXCOUNT = 100000000 results in 1
```

 Important to Start:

Keep in mind that anyone else is able to use a Raspberry Pi. Therefore, please copy all personal data to your personal server area before completing the exercise.

5.2 Race Condition

❖ The C program in Listing 1 is given.

- a) Compile the C program in Eclipse. Alternatively compile it with the command **make**. The corresponding **makefile** has been provided with this exercise.
- b) Run the program a few times. Increase the number of child processes if necessary. What do you notice?
- c) How would you illustrate the behavior, for example, with a diagram?
- d) What should theoretically be done to prevent possible runtime issues?
- e) What is practically possible to prevent such a situation?

Listing 1: uebung05_2_race_posix.c

```
/*
 * race_posix.c
 */
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <stdlib.h>

#include <sys/mman.h>
#include <sys/wait.h>

#define MAXCOUNT 100000000
#define NUM_CHILDREN 40

#define MAXMEMSIZE 1

int main() {
```

```

    int i = 0;                /* counter for loop */
    int shmем_fd = 0;         /* file descriptor of the shared memory segment */
    int *shmем_ptr = NULL; /*
pointer to the shared memory */
    const char name[11] = "/mymemory"; /* name of the shared memory */
    int counter = 0;          /* counter for while - loop */
    int diff = 0;             /* difference between value in shared mem */
    /* and MAXCOUNT */
    int pid[NUM_CHILDREN]; /* array stores the Process IDs of the child proce

/* create a shared memory
*/
shmем_fd = shm_open (name , O_CREAT | O_RDWR , 0644);

if ( shmем_fd < 0)
{
    printf ("Unable to create the shared memory segment.\n");
    perror ("shm_open ") ;
    exit (1) ;
}

/* Adjust the size of the memory segment */
if ( ftruncate ( shmем_fd , MAXMEMSIZE ) < 0)
{
    printf ("Unable to adjust the size.\n");
    perror ("ftruncate");
    exit (1) ;
}

/* attaching the address of the shared memory part to the
 * process' value space */
shmем_ptr = mmap (0, MAXMEMSIZE , PROT_WRITE , MAP_SHARED, shmем_fd, 0)
if ( shmем_ptr < 0)
{
    printf ( "Unable to attach %s.\n",name);
    perror ( "mmap " ) ;
    exit (1) ;
}

*shmем_ptr = 0; /* Init value */

```

```

/* forking NUM_CHILDREN child processes , their PIDs
 * are stored in the pid[]-array
 */
for (i = 0; i < NUM_CHILDREN; i = i + 1)
{
    pid[i] = fork();

    if (pid[i] == -1)
    { /* error checking */
        printf("%d th child not created!\n", i);
        return(0);
    }
    /* every child process has its own count variable.
     * as long as the global counter in shared memory is smaller
     * than MAXCOUNT, this and the proccess' individual count
     * variable is incremented.
     * Thus, the proccess individual count denotes the
     * share on the global count
     */
    if (pid[i] == 0)
    { /* code portion of child process */
        while (*shmemp_ptr < MAXCOUNT)
        {
            *shmemp_ptr = *shmemp_ptr + 1;
            counter = counter + 1;
        }
        printf("%d th child increased value by %d!\n", i, counter);
        /* detaching the address from the process */

        if ( munmap ( shmemp_ptr , MAXMEMSIZE ) < 0)
        {
            printf ( "Unable to detach %s.\n " , name ) ;
            perror ( "munmap" ) ;
            exit (1) ;
        }

        return(0);
    }
}
/* the parent process awaits end of all child processes */
for (i = 0; i < NUM_CHILDREN; i++)
{
    waitpid(pid[i], NULL, 0);
}

```

```
    }
    /* reading the value from shared memory integer and comparing
     * to MAXCOUNT and calculating the difference
     */
    printf("Shared M. = %d - MAXCOUNT = %d", *shmem_ptr, MAXCOUNT);
    diff = *shmem_ptr - MAXCOUNT;
    if (diff > 0)
    {
        printf(" results in %d\n", diff);
    }
    else
    {
        printf("\n");
    }

    /* detaching shared memory */
    if ( munmap (shmem_ptr , MAXMEMSIZE ) < 0)
    {
        printf("Unable to detach %s.\n", name) ;
        perror("munmap") ;
        exit (1) ;
    }
    else
    {
        /* printf ("%s was detached from the process.\n", name); */
    }
    /* close the shared memory
    */
    if ( close( shmem_fd ) < 0) {
        printf("Unable to close %s.\n", name);
        perror("close") ;
        exit (1) ;
    }
    /* remove the shared memory integer finally */
    if ( shm_unlink ( name ) < 0)
    {
        printf("Unable to erase %s.\n", name);
        perror("shm_unlink");
        exit (1) ;
    }

    return 0;
```

Prof. Dr.-Ing. J. Abke

TH Aschaffenburg
Operating Systems and Networks

}